



Assignment Cover Page

CSE: D
Roll no: 46

Name of the student :- <u>Neha Emilin George</u>			
PRN <u>VML2SL5176</u>		Admission No. <u>10302</u>	
Subject Name :- <u>Data Structure And Algorithms</u>		Subject Code:- <u>PCCSI303</u>	
Assignment Title/No : <u>02</u>			
Name of the faculty: <u>Ms. Ambili MA</u>			
Assignment Submitted on <u>29/09/26</u>			
Late submission rules : Max mark will reduced to 50% for 1-5 working day's delay, no mark will be awarded thereafter.			
I am hereby confirming that this assignment is my own and I haven't adopted any unfair means in any steps of its preparation to enhance my performance in this assignment.			
Date : <u>29/09/26</u>		<u>Neha Emilin George</u> <u>Neha Emilin George</u> Sign with Name	
Assignment subdivision	Maximum Mark	Marks awarded	Remarks
A			
B			
C			
<u>Feed back/suggestions :</u>			
Name and sign of the faculty			

Table of Content

- 1). Aim
- 2). Problem statement
- 3). Data structure used
- 4). k-way Merge using Min Heap
- 5). Pseudocode - k-way Merge
- 6). Important Heap states
- 7). Heap Trace Table
- 8). Final Merged List
- 9). Pairwise Merging
- 10). Pseudocode
- 11). First pairwise Merge - Trace Table
- 12). Second pairwise Merge - Trace Table
- 13). Operation Count
- 14). Analyse and Comparison
- 15). Heap Size Analysis
- 16). Time Complexity
- 17). No of Comparisons
- 18). Space Complexity
- 19). Comparison Table
- 20). Advantages of Min Heap and pairwise Approach
- 21). Limitations
- 22). Conclusion
- 23). Result
- 24). Program and output

10) A financial system receives three already sorted transaction lists:

$L_1 = 10, 30, 50, 70$

$L_2 = 20, 40, 60, 80$

$L_3 = 15, 35, 55, 75$

a) Represent the three lists using suitable data structure and implement a k-way merge using a Min Heap. Show the important a k-way merge using a Min Heap Show the important heap states during execution

b) Implement a single pairwise merging approach for the same lists. Execute both approaches and record the no of major operations.

c) Analyse and compare: Heap size

No of comparisons, Time complexity

Space requirement

Determine which approach is more stable when the no of sorted files increases

• Aim

To implement and compare two approaches for merging three sorted transaction lists

1) k-way merge using a Min Heap

2) pairwise merging using the simple two-way merge technique

A) A financial system receives 3 already sorted transaction lists

- $L_1 = 10, 30, 50, 70$

The tasks are to

- $L_2 = 20, 40, 60, 80$

- Represent the list using suitable data structure

- $L_3 = 15, 35, 55, 70$

- implement k-way merging using a

- Min Heap
- implement pairwise merging.

- Record the execution steps and compare both

- Determine which approach is more suitable

3). Data Structure Used

Data Structure

Purpose

Arrays

Store the three sorted input lists and the final merged list

MinHeap

Maintain the smallest current element among the lists

Heap node

Stores the value, source list number, and position of an element

Temporary array

Stores intermediate results during pairwise merging

• A). k-WAY Merge Using a Min Heap

Pseudocode : INSERT INTO MIN HEAP

INSERT(heap, node)

1. Add node at the end of the heap.

2. Set i to the index of the inserted node

3. while $i > 0$:

$$\text{Parent} = (i-1)/2$$

IF $\text{heap}[\text{Parent}].\text{value} \leq \text{heap}[i].\text{value}$:

stop

Swap $\text{heap}[\text{Parent}]$ and $\text{heap}[i]$

$i = \text{Parent}$

4. RETURN

B). Pseudo Code - Delete Minimum

EXTRACT-MIN (heap)

1). store the root element as min

2). Replace the root with the last element

3). Reduce heap size by 1

4). Set $i \leftarrow 0$

5). while i has a child:

- i. Find the left and right child

- ii. Select the smaller child

- iii. if current value $\leq$ smaller child value, stop

- iv. Otherwise, swap the current node with the smaller child

- v. Set i to the child position

6). Return min

C) Pseudocode - k-way Merge

k-way Merge (L_1, L_2, L_3)

1). create an empty Min Heap

2). Insert the first element of L_1

3). Insert the first element of L_2

4). Insert the first element of L_3

5). Create an empty output array

6). while heap is not empty

- i) Extract the minimum element from the heap
- ii) Add the extracted element to the output array
- iii) Identify its source list
- iv) if the source list contains another element, insert the next element into the heap
- v). Restore the Min heap property

7. Display the merged list

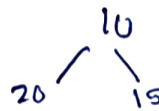
8. stop

• Important Heap states

initial insertion

L1 → 10
L2 → 20
L3 → 15

Initial Min Heap



Array representation: [10 20 15]

• Heap Trace Table

Step	Element Removed	Next element	Heap after operation
Initial	—	10, 20, 15	[10, 20, 15]
1	10	30	[15, 20, 30]
2	15	35	[20, 30, 35]
3	20	40	[30, 35, 40]
4	30	50	[35, 40, 50]
5	35	55	[40, 50, 55]
6	40	60	[50, 55, 60]
7	50	70	[55, 60, 70]
8	55	75	[60, 70, 75]
9	60	80	[70, 75, 80]
10	70	—	[75, 80]
11	75	—	[80]
12	80	—	[]

- Final merged list

10, 15, 20, 30, 35, 40, 50, 55, 60, 70, 75, 80

B) Pairwise Merging

In pairwise merging, two sorted lists are merged at a time

$$L_1 + L_2 = (L_1 + L_2) + L_3$$

+ hencefore $L_1 + L_2 \rightarrow M_1$

$$M_1 + L_3 \rightarrow \text{Final}$$

Pseudocode FOR PAIRWISE MERGING

PAIRWISE-MERGE (L_1, L_2, L_3)

1). Merge L_1 and L_2 into M_1

2). Merge M_1 and L_3 into M_2

3). Return M_2

Merge of two lists

MERGE (A, B)

1. set $i=0, j=0$ and $k=0$

2. Compare $A[i]$ and $B[j]$

3. copy the smaller element into the result

4. move the corresponding index forward

5. Repeat until one list becomes empty

6. copy all remaining elements from the other list

7. Return the merged list

* First Pairwise Merge

Merge:

$L_1 = 10, 30, 50, 70$

$L_2 = 20, 40, 60, 80$

Trace

Step	L1 element	L2 element	Selected
1	10	20	10
2	30	20	20
3	30	40	30
4	50	40	40
5	50	60	50
6	70	60	60
7	70	80	70
8	-	80	80

Result $M_1 = 10\ 20\ 30\ 40\ 50\ 60\ 70\ 80$

No of Comparisons 7

* Second Pairwise Merge

Merge

M_1 and L_3

$M_1 = 10\ 20\ 30\ 40\ 50\ 60\ 70\ 80$

$L_3 = 15\ 35\ 55\ 75$

Steps	Min Element	Max Element	Selected
1	10	15	10
2	20	15	15
3	20	35	20
4	30	35	30
5	40	35	35
6	40	55	40
7	50	55	50
8	60	55	55
9	60	75	60
10	70	75	70
11	80	75	75
12	80	-	80

No. of comparison = 11

Result : 10 15 20 30 35 40 ~~50~~ 55 60 70 75 80

* Operation Count

First merge : $4 + 4 - 1 = 7$ comparison

Second : $2 + 4 - 1 = 5$ comparison

Total : $7 + 5 = 12$

Min Heap Method

Initial insertions = 3

Extractions = 12

additional insertion = 9

∴ Total insert operation = 12

Total extract-min operation = 12

the heap never contain more than $k=3$

C). Analyse and Compare:

1) Heap Size Analysis

For k -way merging the heap contains at most one active element from each list

$\therefore$ Maximum Heap size = k

here $k=3$ hence: maximum Heap size = 3

Heap size during execution:

$3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

2) Time Complexity

k -way Merge Using Min Heap

Let N = total no of elements

k = no of sorted lists

each element is processed using the Min Heap

the heap operation takes : $O(\log k)$

$\therefore$ Time complexity = $O(N \log k)$

For this problem $k=3$
 $N=12$ $\therefore O(12 \log 3)$

3) Number of Comparisons

Pairwise Merge

• First merge ($L_1 + L_2$) = 7 Comparisons

• Second merge (result + L_3) = 11 Comparisons

Total = 18 comparisons

4). Space Complexity

Method Space Complexity

Min Heap $O(k)$

pairwise Merge $O(N)$

here k = no of lists and N = total no of elements

Comparison Table

Parameters	k-way Merge Using Min Heap	Simple pairwise Merge
Inputs lists	3	3
Total element	12	12
Main data structure	Min Heap	Arrays
Max heap size	3	not applicable
Extract operations	12	not applicable
insert operation	12 total	not applicable
element comparison for given pairwise execution	Depends on heap implementation	18
Overall time complexity	$O(N \log k)$	Can approach $O(Nk)$ for Sequential merging
Auxiliary Space	$O(k)$ heap	$O(N)$ intermediate
Implementation complexity	Moderate	Simple

* Advantages of MIN HEAP APPROACH

- 1). Efficiently finds the smallest current element
- 2). works well with a large no of sorted list
- 3). Requires only one active element from each list in the heap
- 4). Has time complexity $O(n \log k)$
- 5). Heap space is only $O(k)$
- 6). it is suitable for external sorting and merging multiple sorted files

* Advantages of Pairwise approach

- 1). simple to understand
- 2). Easy to implement
- 3). Does not require a heap
- 4). suitable when only a small number of lists need to be merged
- 5). The merge of two sorted lists itself is efficient

* Limitations.

Min Heap

- implementation is slightly more complex
- Heap operations require additional processing
- The heap structure must be maintained correctly

Pairwise

- Intermediate lists can become large

- elements may be processed repeatedly
- Sequential merging becomes less efficient as the number of files increases

Conclusion

Both k-way Min Heap and pairwise merging successfully merged the three sorted lists.

The final output is: 10 15 20 30 35 40 50 55
60 70 75 80

The pairwise method is simple and performed 18 comparisons for the given input. The Min Heap method has a maximum heap size of 3 and a time complexity of $O(n \log k)$ making it more suitable when the number of sorted files

Result

Both methods were successfully implemented in C and produced the correct sorted transaction list. The experiment demonstrated efficient merging using a Min Heap and compared it with pairwise Merging.

A) C Program — K-Way Merge Using Min Heap

```
#include <stdio.h>

#define K 3
#define MAX 20
#define N 12

int L[K][4] = {
    {10, 30, 50, 70},
    {20, 40, 60, 80},
    {15, 35, 55, 75}    };

typedef struct {
    int value;
    int list;
    int pos;
} Node;

Node heap[MAX];
int size = 0;
int comparisons = 0;

void swap(Node *a, Node *b) {
    Node temp = *a;
    *a = *b;
    *b = temp;    }

void insert(Node x) {
    int i = size++;
    heap[i] = x;
    while (i > 0) {
        int p = (i - 1) / 2;
        comparisons++;
```

```

        if (heap[p].value <= heap[i].value)
            break;
        swap(&heap[p], &heap[i]);
        i = p;
    }
}

```

```

Node extractMin() {
    Node result = heap[0];
    heap[0] = heap[--size];
    int i = 0;
    while (2 * i + 1 < size) {
        int left = 2 * i + 1;
        int right = left + 1;
        int smallest = left;
        if (right < size) {
            comparisons++;
            if (heap[right].value < heap[left].value)
                smallest = right;
        }
        comparisons++;
        if (heap[i].value <= heap[smallest].value)
            break;
        swap(&heap[i], &heap[smallest]);
        i = smallest;
    }
    return result;
}

```

```

void displayHeap() {
    printf("\n");
    for (int i = 0; i < size; i++) {
        printf("%d", heap[i].value);
    }
}

```

```

    if (i < size - 1)
        printf(", ");
    printf("]\n");
}

int main() {
    int output[N];
    int count = 0;
    printf("Initial Lists:\n");
    for (int i = 0; i < K; i++) {
        printf("L%d: ", i + 1);
        for (int j = 0; j < 4; j++)
            printf("%d ", L[i][j]);
        printf("\n");
    }
    for (int i = 0; i < K; i++) {
        Node x = {L[i][0], i, 0};
        insert(x);
    }
    printf("\nInitial Heap: ");
    displayHeap();
    printf("\nK-Way Merge Trace:\n");
    while (size > 0) {
        Node x = extractMin();
        output[count++] = x.value;
        printf("Extracted: %d (L%d)\n",
            x.value, x.list + 1);
        int nextPos = x.pos + 1;
        if (nextPos < 4) {
            Node next = {

```

```

        L[x.list][nextPos],
        x.list,
        nextPos };
    insert(next);    }
    printf("Heap: ");
    displayHeap(); }
    printf("\nMerged Output: ");
    for (int i = 0; i < count; i++)
        printf("%d ", output[i]);
    printf("\nTotal elements: %d\n", count);
    printf("Heap key comparisons: %d\n", comparisons);
    return 0;
}

```

Initial Lists:

L1: 10 30 50 70

L2: 20 40 60 80

L3: 15 35 55 75

Initial Heap: [10, 20, 15]

K-Way Merge Trace:

Extracted: 10 (L1)

Heap: [15, 20, 30]

Extracted: 15 (L3)

Heap: [20, 30, 35]

Extracted: 20 (L2)

Heap: [30, 35, 40]

Extracted: 30 (L1)

Heap: [35, 40, 50]

Extracted: 35 (L3)

Heap: [40, 50, 55]

Extracted: 40 (L2)

Heap: [50, 55, 60]

Extracted: 50 (L1)

Heap: [55, 60, 70]

Extracted: 55 (L3)

Heap: [60, 70, 75]

Extracted: 60 (L2)

Heap: [70, 75, 80]

Extracted: 70 (L1)

Heap: [75, 80]

Extracted: 75 (L3)

Heap: [80]

Extracted: 80 (L2)

Heap: []

Merged Output: 10 15 20 30 35 40 50 55 60 70 75 80

Total elements: 12

Heap key comparisons: 21

10 B)

Pairwise Merging of Sorted Lists in C

```
#include <stdio.h>
```

```
void merge(int A[], int n, int B[], int m, int C[])
```

```
{
```

```
    int i = 0, j = 0, k = 0;
```

```
    while (i < n && j < m)
```

```
    {
```

```
        if (A[i] <= B[j])
```

```
            C[k++] = A[i++];
```

```
        else
```

```
            C[k++] = B[j++];
```

```
    }
```

```
    while (i < n)
```

```
        C[k++] = A[i++];
```

```
    while (j < m)
```

```
        C[k++] = B[j++];
```

```
}
```

```
int main()
```

```
{
```

```
    int L1[] = {10, 30, 50, 70};
```

```
    int L2[] = {20, 40, 60, 80};
```

```
    int L3[] = {15, 35, 55, 75};
```

```
    int temp[8];
```

```
    int result[12];
```

```
    int i;
```

```
    merge(L1, 4, L2, 4, temp);
```

```
printf("After merging L1 and L2:\n");  
for (i = 0; i < 8; i++)  
    printf("%d ", temp[i]);  
merge(temp, 8, L3, 4, result);  
printf("\n\nFinal merged list:\n");  
for (i = 0; i < 12; i++)  
    printf("%d ", result[i]);  
return 0;  
}
```

After merging L1 and L2:
10 20 30 40 50 60 70 80

Final merged list:

10 15 20 30 35 40 50 55 60 70 75 80

Process returned 0 (0x0) execution time : 4.922 s